

Deep Learning I

Digitization & Data Analytics — Lecture 8

Complete exam-oriented study guide: annotated notes, redrawn diagrams, worked examples, and a research-grade Q&A bank (conceptual, computational, and past-exam style) covering the single-layer perceptron through backpropagation and generalization.

[Single-Layer Perceptron](#)

[Multi-Layer Perceptrons](#)

[Backpropagation](#)

[Regularization](#)

Contents

1	Foundations: What is Deep Learning?	3
2	The Single-Layer Perceptron (1943–1969)	4
3	The XOR Problem & Linear Separability	6
4	Multi-Layer Perceptrons (MLPs)	7
5	Activation Functions	8
6	Loss Functions & Gradient Descent	9
7	Backpropagation — Theory & Worked Examples	10
8	Learning Rate, Overfitting & Regularization	12
9	Training vs. Inference & Hardware	13
10	Exam-Style Question Bank (30 Q&A, graded by difficulty)	14
11	Formula & Definition Cheat Sheet	19

How to use this guide: Each section mirrors the lecture structure (Introduction → Single-Layer → Multi-Layer). Diagrams are redrawn for clarity. Every section ends with a short self-check Q&A; Section 10 contains a large, exam-style question bank spanning all topics — ideal for last-minute review or for constructing your own flashcards.

1. Foundations: What is Deep Learning?

1.1 AI $\supset$ ML $\supset$ DL

Artificial Intelligence (AI) is the broadest field: any technique that lets machines mimic aspects of human intelligence (rule-based systems, search, planning, learning).

Machine Learning (ML) is the subset of AI where the system *learns* patterns from data rather than following hand-coded rules.

Deep Learning (DL) is the subset of ML that uses **multi-layered artificial neural networks** to automatically learn hierarchical feature representations directly from raw data (pixels, audio waveforms, text tokens), removing the need for manual feature engineering.

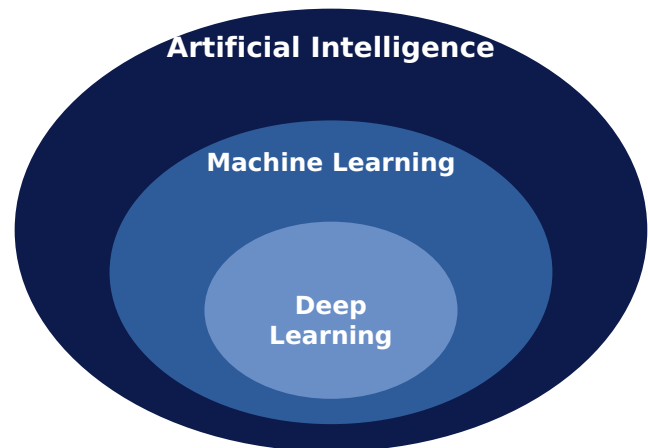


Fig. 1.1 — Nested relationship AI $\supset$ ML $\supset$ DL

1.2 Biological Inspiration

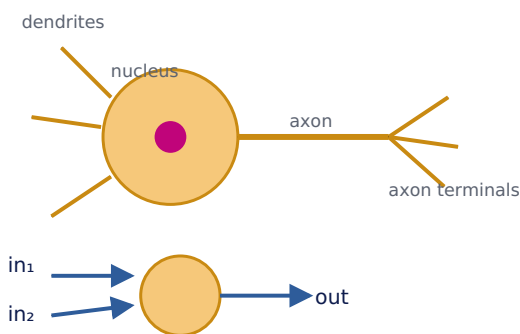


Fig. 1.2 — Biological neuron (top) vs. its computational abstraction (bottom): dendrites $\rightarrow$ weighted inputs, axon $\rightarrow$ output.

Artificial neurons are a loose mathematical abstraction of biological neurons: dendrites receive many signals, the cell body aggregates them, and if the aggregate exceeds a threshold, an electrical pulse ("spike") travels down the axon to other neurons. In the artificial analogue, **inputs** are multiplied by **weights**, summed, and passed through an **activation function** that plays the role of the firing threshold.

This is only a loose analogy — real biological neurons are vastly more complex (spike timing, neurotransmitter chemistry, plasticity). The artificial neuron is a simplified computational model, not a simulation of biology.

1.3 Typical Use Cases

Domain	Examples
Computer Vision	Autonomous vehicles (object/lane detection), medical imaging (tumor detection), face recognition, quality inspection
Speech Recognition	Voice assistants (Siri, Alexa), accessibility tools (speech-to-text captioning)
Natural Language Processing	Chatbots, sentiment analysis, machine translation, large language models

Q1.1 — Why is Deep Learning considered a subset of Machine Learning rather than a separate field?

Because DL still follows the ML paradigm of learning a function from data (rather than hand-coded rules); what distinguishes it is the use of deep (multi-layer) neural network architectures that learn hierarchical representations automatically, instead of relying on hand-engineered features as classical ML often does.

2. The Single-Layer Perceptron (1943-1969)

- **1943 — McCulloch & Pitts**, "A Logical Calculus of the Ideas Immanent in Nervous Activity." Introduced the first mathematical neuron model: a **linear threshold unit** with **binary output** (0 or 1). It could represent logical functions but **could not learn its weights** — they had to be set by hand.
- **1957/58 — Frank Rosenblatt**, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain." Introduced the first algorithm capable of **learning** weights from data (the Perceptron Learning Rule), starting from random weights and updating them only on misclassification.
- **1969 — Minsky & Papert**, "Perceptrons." Proved formally that a single perceptron can only solve **linearly separable** problems, famously highlighting the **XOR problem** as something a single-layer perceptron cannot represent. This contributed to the first "AI winter."

2.1 The McCulloch-Pitts Neuron

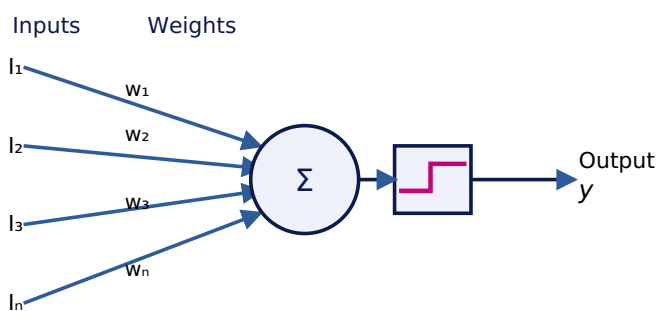


Fig. 2.1 — McCulloch-Pitts neuron: weighted sum → hard threshold → binary output.

- Computes $\Sigma = \sum_i I_i \cdot w_i$
- Applies a **hard threshold**: output is 1 if the sum exceeds a fixed threshold, else 0
- Weights are fixed by design — **no learning** mechanism
- Historically important as proof that networks of simple binary units could in principle compute any logical proposition

2.2 The Perceptron Learning Rule (Rosenblatt)

$$\omega_i^{(n+1)} \leftarrow \omega_i^{(n)} + \alpha \cdot (y_i - \theta_i) \cdot x_i$$

Symbol	Meaning
α	learning rate (fixed step size, e.g. 0.3)
y_i	true/target label for sample i
θ_i	the perceptron's current output (prediction) for sample i
x_i	the input feature value that the weight ω_i multiplies

Key property: weights are updated *only* when the sample is misclassified ($y \neq \text{prediction}$); correctly classified samples leave the weights unchanged. This is the earliest form of **error-driven learning**.

2.3 Worked Example — Training an AND-like Perceptron

Setup: $\alpha = 0.3$, initial weights $\omega_1 = \omega_2 = 1$, activation $f(\Sigma) = 1$ if $\Sigma > 0.4$, else 0, target function = logical AND.

if $\Sigma > 0.4$, else 0, target function = logical AND.

x_1	x_2	y (target)
0	0	0
0	1	0
1	0	0
1	1	1

Sample 1 (0,0): $\Sigma=0 \rightarrow f(\Sigma)=0 = y$ ✓ (no update)

Sample 2 (0,1): $\Sigma = 0 \cdot 1 + 1 \cdot 1 = 1 \rightarrow f(\Sigma)=1, y=0 \rightarrow \text{error } \epsilon = y - f(\Sigma) = -1$

Update: $\omega_1 \leftarrow 1 + 0.3(-1)(0) = 1.0$; $\omega_2 \leftarrow 1 + 0.3(-1)(1) = 0.7$

Training continues, cycling through all 4 samples repeatedly, updating only on mismatches, **until an entire pass produces zero updates** (convergence). The final table in the lecture shows weights settling near $\omega_1 \approx 0.4, \omega_2 \approx 0.4$, correctly separating the AND truth table.

Exam tip: The perceptron convergence theorem guarantees this update rule finds a separating hyperplane *in finite steps* — but **only if the data is linearly separable**. If it is not (e.g., XOR), the algorithm never converges.

Q2.1 — What exactly did Rosenblatt's 1958 contribution add on top of the 1943 McCulloch-Pitts neuron?

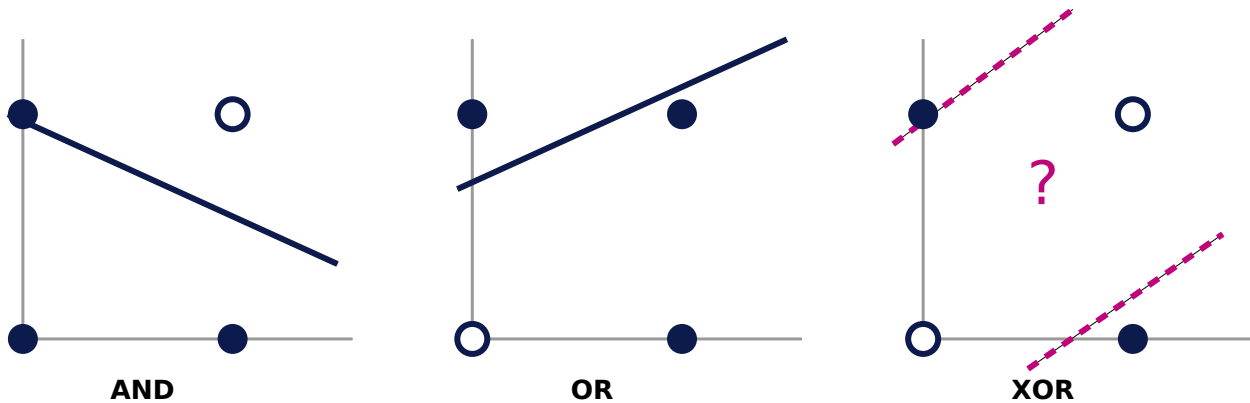
The neuron's mathematical form (weighted sum + threshold) was already known; Rosenblatt's key contribution was the **learning algorithm** — a rule to automatically adjust weights from labeled examples via an error-driven update, rather than requiring weights to be hand-designed.

Q2.2 — In the perceptron update rule, why is no update applied when the sample is correctly classified?

Because the error term $(y - \theta) = 0$ when the prediction already matches the target, making the whole update term zero — the rule is explicitly designed to only correct mistakes, leaving already-correct weights untouched.

3. The XOR Problem & Linear Separability

A single perceptron computes a **linear decision boundary** (a line in 2D, a hyperplane in general). It can therefore only correctly classify data that is **linearly separable** — i.e., one straight line can perfectly split the two classes.



I_1	I_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Why XOR fails: No single straight line can separate the "1" outputs (0,1) & (1,0) from the "0" outputs (0,0) & (1,1) — the positive class occupies two diagonally opposite corners. This requires at least **two** decision lines / a non-linear (piecewise) boundary — i.e. a **multi-layer** network with a hidden layer.

Q3.1 — Why can a hidden layer solve the XOR problem when a single perceptron cannot?

A hidden layer lets the network first compute intermediate linear boundaries (e.g., one hidden unit approximating OR, another approximating NAND) and then combine these intermediate results linearly in the output layer (essentially AND-ing them). This composition of linear boundaries plus a non-linear activation creates a non-linear, piecewise decision surface, which can represent XOR.

Q3.2 — Historically, why did the XOR limitation cause an "AI winter"?

Minsky & Papert's 1969 book rigorously proved this limitation of the single-layer perceptron, and — crucially — no efficient algorithm was yet known to train multi-layer networks. This led to a sharp drop in funding and research interest in neural networks until backpropagation (1986) provided a practical training method for multi-layer networks.

4. Multi-Layer Perceptrons (MLPs)

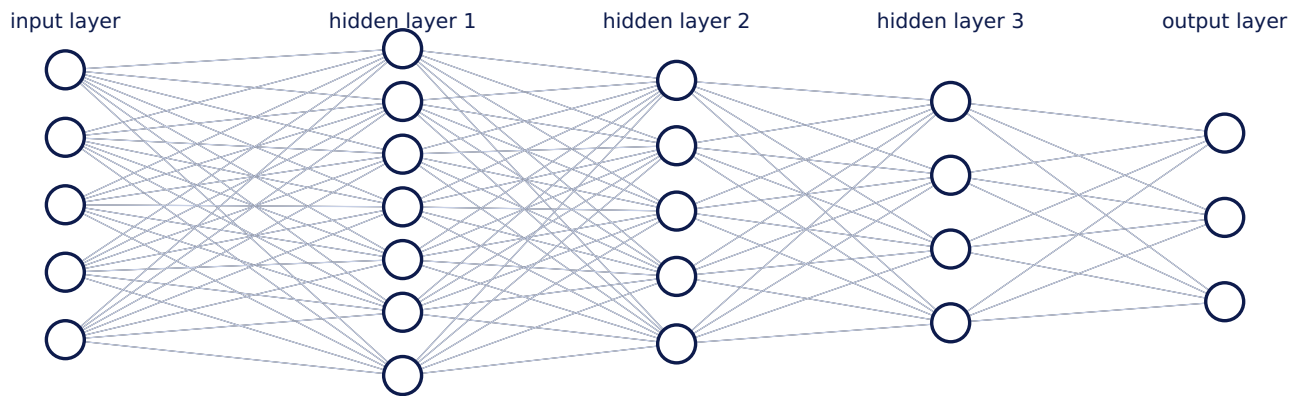


Fig. 4.1 — A deep, fully-connected feedforward network (Multi-Layer Perceptron): every neuron connects to every neuron of the adjacent layers.

4.1 Anatomy of a Single Neuron in the Network

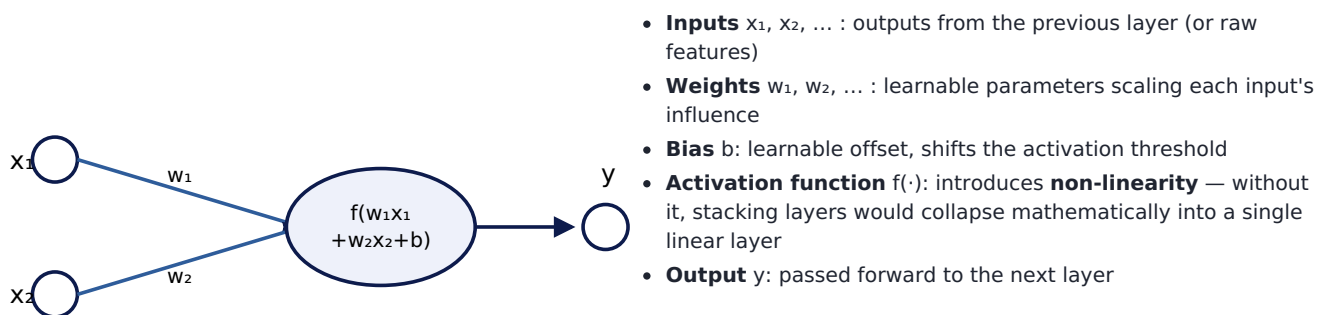


Fig. 4.2 — A single MLP neuron: linear combination + bias, then a non-linear activation f .

4.2 Core Definitions

- MLPs are **feedforward** networks: information flows strictly forward, from input to output, with no cycles.
- Layers are typically **fully connected (dense)**: every neuron connects to all neurons in the adjacent layer.
- The **output layer** produces the network's final prediction $\hat{y}$; its size/activation depends on the task (e.g., 1 sigmoid unit for binary classification, softmax over C units for C -class classification, linear unit for regression).
- Training uses **supervised learning** with a labeled dataset $X = \{(x_i, y_i)\}_{i=1}^N$, where each $x_i = (x_{i1}, \dots, x_{ip})$ is a p -dimensional input and y_i its label.
- A **loss function** L compares prediction $\hat{y}$ to true label y , quantifying "how wrong" the network is.

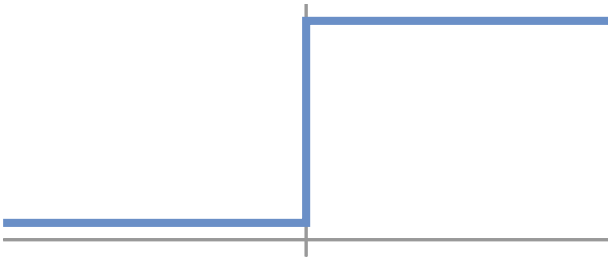
Q4.1 — Why must the activation function be non-linear?

If every layer used only a linear (or absent) activation, the entire composition of layers would mathematically reduce to a single linear transformation, no matter how many layers you stack (since a linear function of a linear function is still linear). Non-linear activations are what let deep networks represent complex, non-linear decision boundaries such as XOR.

5. Activation Functions

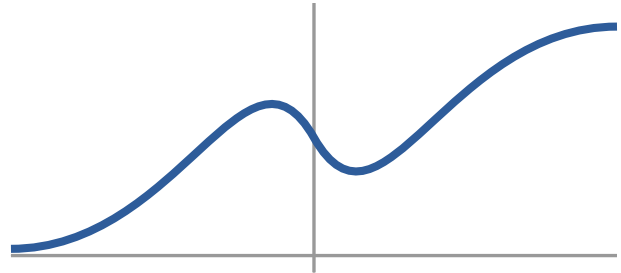
Threshold (Step) Function

$$f(x) = 1 \text{ if } \sum w_i x_i + b > 0, \text{ else } 0$$



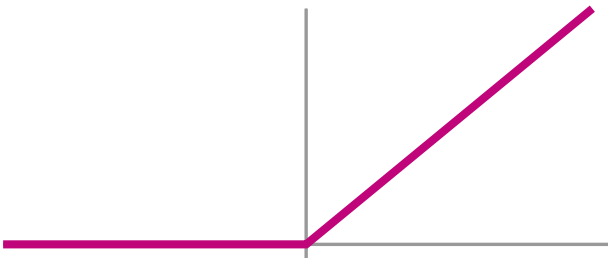
Sigmoid / Logistic

$$f(x) = 1 / (1 + e^{-\beta x})$$



ReLU

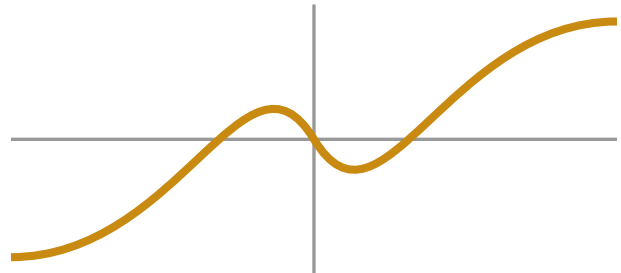
$$f(x) = \max(0, x)$$



Output range (0,1) — interpretable as a probability. Suffers from **vanishing gradients** for large $|x|$.

Tanh

$$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$



Zero-centered version of sigmoid, output range $(-1, 1)$. Also others exist: Leaky ReLU, GELU, ELU, Swish, ...

Function	Range	Pros	Cons
Threshold	{0,1}	Simple, historically first	Not differentiable → cannot use gradient-based learning
Sigmoid	(0,1)	Smooth, probabilistic interpretation	Vanishing gradients, not zero-centered, expensive $\exp()$
Tanh	$(-1,1)$	Zero-centered, smooth	Still saturates (vanishing gradients) at extremes
ReLU	$[0, \infty)$	Cheap, mitigates vanishing gradients, sparse activations	"Dying ReLU" (neurons stuck outputting 0), unbounded

Q5.1 — Why did the field move away from the threshold function toward smooth activations like sigmoid/tanh, and later to ReLU?

The threshold function has zero derivative almost everywhere (and is undefined at 0), which makes gradient-based learning (backpropagation) impossible. Smooth functions like sigmoid/tanh provide usable gradients everywhere, enabling gradient descent. However, sigmoid/tanh saturate for large inputs, causing vanishing gradients in deep networks; ReLU was later adopted because its gradient is a constant 1 for positive inputs, which mitigates vanishing gradients and is computationally cheap.

6. Loss Functions & Gradient Descent

Training means finding parameters $\mathbf{w}$ that make the network's predictions $\hat{y}$ as close as possible to the true targets y , according to some **loss function** L . This is framed as an optimization problem: minimize $L(y, X, \mathbf{w})$ over $\mathbf{w}$.

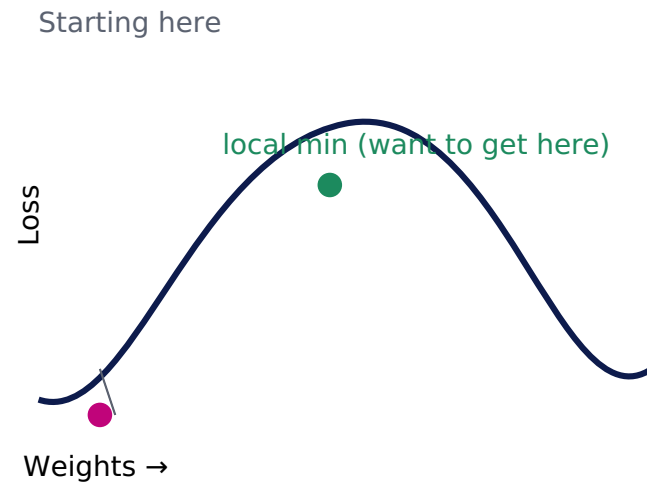


Fig. 6.1 — Loss landscape: gradient descent walks downhill along the negative gradient toward a (local) minimum.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \cdot \nabla_{\mathbf{w}} L(y, X, \mathbf{w})$$

- **Gradient Descent** (Cauchy, 1847) — repeatedly step in the direction opposite to the gradient of the loss, since the gradient points in the direction of steepest *increase*.
- α is again the **learning rate**, controlling step size.
- Loss over the full dataset decomposes into a **sum over samples**:

$$\nabla_{\mathbf{w}} L(\mathbf{y}, X, \mathbf{w}) = \sum_i \nabla_{\mathbf{w}} L(\mathbf{y}_i, \mathbf{x}_i, \mathbf{w})$$
 — the gradient w.r.t. the whole dataset is the sum of per-sample gradients.
- In practice, the objective is often the **negative log-likelihood**: $\nabla_{\mathbf{w}} \sum_i -\log p_{\text{model}}(y_i | x_i, \mathbf{w})$.

Chain rule (Leibniz, 1673): $d/dx f(g(x)) = (df/dg)(dg/dx)$. **Multivariate chain rule:** $d/dx f(g_1(x), \dots, g_M(x)) = \sum_i (\partial f / \partial g_i)(dg_i/dx)$. This is the mathematical engine behind backpropagation (Section 7).

Q6.1 — What is the difference between the gradient "with respect to the entire dataset" and "with respect to a single data point," and why does this matter in practice?

The full-dataset gradient is the sum of the per-sample gradients over all N training examples. Computing it exactly (as in "batch" gradient descent) is accurate but expensive for large N . In practice, deep learning uses **mini-batch** or stochastic gradient descent, approximating the full gradient using only a small random subset (batch) of samples at each step — this is both computationally necessary (since large datasets don't fit in GPU memory) and, empirically, often improves generalization due to the added noise.

7. Backpropagation — Theory & Worked Examples

1986 — Rumelhart, Hinton & Williams, "Learning representations by back-propagating errors" (Nature). The paper describes a procedure that repeatedly adjusts the weights of connections in a network to minimize the difference between the network's actual output and the desired output, by efficiently applying the (multivariate) chain rule: values are computed **forward**, gradients are computed **backward**.

7.1 The Core Idea

Forward pass: compute intermediate values layer by layer until the loss L is obtained. Backward pass: starting from $\partial L / \partial L = 1$, propagate **local gradients** backward through each computational node using the chain rule, accumulating **back-propagated gradients** at each node — all the way back to every weight in the network.

7.2 Worked Example 1 — Simple Chain

Forward: (1) $y = x^2$, (2) $L = 2y \implies L = 2x^2$

Backward:

(2) $\partial L / \partial y = \partial L / \partial L \cdot \partial L / \partial y = 1 \cdot 2 = 2$

(1) $\partial L / \partial x = \partial L / \partial y \cdot \partial y / \partial x = 2 \cdot 2x = 4x$

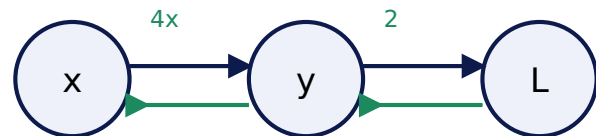


Fig. 7.1 — Forward pass (blue, $\rightarrow$) and backward pass (green, $\leftarrow$) for $L = 2x^2$.

7.3 Worked Example 2 — Branching Graph

Forward: (1) $y = y(x)$, (2) $u = u(y)$, $v = v(y)$, (3) $L = L(u, v)$. When a node **feeds into two downstream paths** (here, y feeds both u and v), the multivariate chain rule requires that **all incoming gradients be summed**:

$$d/dy \cdot L(u(y), v(y)) = (\partial L / \partial u) \cdot (du/dy) + (\partial L / \partial v) \cdot (dv/dy)$$

Step	Backward computation
(3)	$\partial L / \partial u = \partial L / \partial L \cdot \partial L / \partial u = \partial L / \partial u$
(3)	$\partial L / \partial v = \partial L / \partial L \cdot \partial L / \partial v = \partial L / \partial v$
(2)	$\partial L / \partial y = (\partial L / \partial u)(\partial u / \partial y) + (\partial L / \partial v)(\partial v / \partial y) \leftarrow \text{gradients summed}$
(1)	$\partial L / \partial x = (\partial L / \partial y)(\partial y / \partial x)$

Common exam trap: forgetting to **sum** gradients when a node has multiple outgoing edges (fan-out) during the forward pass. Every unique path from a parameter to the loss contributes an additive term to that parameter's gradient.

Q7.1 — Why is the reuse of "local gradients" during backpropagation computationally efficient compared to computing each weight's gradient from scratch?

Backpropagation reuses the already-computed downstream gradient ($\partial L / \partial [\text{downstream node}]$) and multiplies it by the local derivative ($\partial [\text{downstream}] / \partial [\text{current node}]$) — this means the cost of computing all gradients in the network is proportional to just one extra forward-pass-worth of computation (dynamic programming style reuse), rather than growing exponentially with network depth as naive symbolic differentiation would.

Q7.2 — What historical/mathematical tool from 1673 underlies the entire backpropagation algorithm developed in 1986?

The Leibniz chain rule for differentiating compositions of functions. Backpropagation is essentially a systematic, efficient (reverse-mode) application of the multivariate chain rule to the computational graph defined by a neural network's forward pass.

8. Learning Rate, Overfitting & Regularization

8.1 Learning Rate



Too low: tiny steps, very slow convergence

Just right: efficient, steady convergence

Too high: overshoots, may diverge/oscillate

Empirically, the optimal learning rate lies in a narrow "sweet spot": too low → loss barely improves; too high → loss oscillates or explodes (diverges). Practitioners often use a **learning-rate range test** or schedules (decay, warm-up, adaptive optimizers like Adam) to find/adjust it during training.

8.2 Overfitting vs. Underfitting

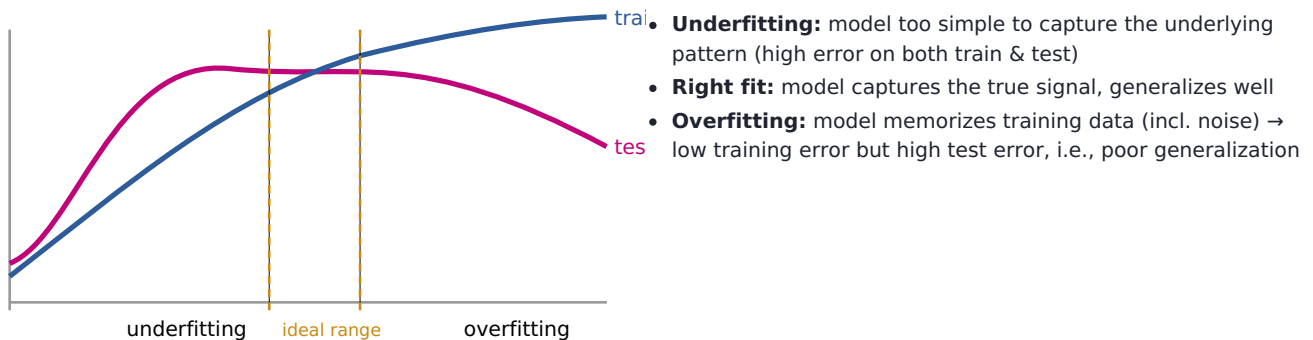


Fig. 8.1 — Bias-variance / model-complexity trade-off: training error keeps falling, but test error rises again once the model starts memorizing noise (overfitting).

8.3 Preventing Overfitting

Technique	How it helps
More training data	Harder to memorize noise when there are more, more diverse examples
Data augmentation	Synthetically expands the training set (rotations, crops, noise, ...) to improve generalization
Dropout	Randomly deactivates neurons during training, preventing co-adaptation of features
Early stopping	Monitor validation loss during slow, iterative gradient-descent fitting; stop once it starts increasing (overfitting detected)
Regularization (L1/L2)	Adds a penalty term to the loss that discourages large weights, trading a little training accuracy for better generalization (L2 = weight decay / ridge / Tikhonov regularization)

Definition (Goodfellow et al., 2016): Regularization is any modification to a learning algorithm intended to reduce its generalization error but not its training error.

Q8.1 — Why does an overly high learning rate risk divergence rather than just slower convergence?

A very large step can overshoot the minimum entirely, landing on a point with even higher loss than before; repeating this can cause the parameters to oscillate with ever-increasing amplitude around (or away from) the minimum, so the loss grows without bound instead of shrinking — that's divergence.

Q8.2 — What is the essential distinction between "reducing training error" and "reducing generalization error," and why does regularization deliberately not target the former?

Training error measures fit to the seen data; generalization error measures performance on new, unseen data — the actual goal of learning. A model can achieve near-zero training error by pure memorization while generalizing poorly. Regularization intentionally constrains the model (e.g., penalizing large weights) so that it does *not* perfectly fit training data, trading a bit of training accuracy for a model that generalizes better — that's precisely why regularization can even slightly increase training

error while decreasing test error.

9. Training vs. Inference & Hardware

Training

- Uses a large labeled dataset
- Runs both a **forward pass** (produce prediction, compare to label) and a **backward pass** (compute gradients, update weights)
- Computationally expensive — repeated many times (epochs) over large data

Inference

- Uses new, unseen (often smaller and more varied) input
- Only a **forward pass** is needed — no gradient computation or weight update
- Comparatively cheap, and must often run in real time (e.g., in a deployed product)

- Large datasets often do not fit entirely into GPU memory → training uses **mini-batches**: $X_{\text{batch}} \subset X$, $X_{\text{batch}} < X$ in size.
- **GPUs** are one major reason deep learning became practical: they are extremely efficient at the large matrix multiplications that dominate neural network computation, enabling training of large networks on large datasets in feasible time.

Rule of thumb: **training** $\gg$ **inference** in compute cost. This asymmetry is why cloud providers offer separate "training" vs. "inference" hardware/pricing tiers, and why techniques like quantization/distillation exist specifically to make inference cheaper for deployment.

Q9.1 — Why is a mini-batch used instead of feeding the entire dataset through the network at once?

Large datasets frequently exceed available GPU memory, so only a subset (mini-batch) can be processed at a time; additionally, mini-batch gradient estimates introduce beneficial stochastic noise that can help escape poor local minima/saddle points and often improves generalization compared to full-batch gradient descent.

10. Exam-Style Question Bank

30 questions in the style typically seen in past-year exams and problem sheets for this course — mixing conceptual (**easy**), applied/computational (**medium**), and derivation/comparison (**hard**) questions. Use these to self-test after reviewing Sections 1–9.

1. **easy Define Artificial Intelligence, Machine Learning, and Deep Learning, and state their nested relationship.**

AI: machines mimicking intelligent behavior generally. ML: subset of AI where systems learn patterns from data instead of explicit rules. DL: subset of ML using deep (multi-layer) neural networks to learn hierarchical representations automatically. Relationship: $AI \supset ML \supset DL$.

2. **easy What two limitations did the 1943 McCulloch-Pitts neuron have?**

(1) It used a hard binary threshold, so its output was strictly 0 or 1 (not differentiable/continuous); (2) it had no learning mechanism — weights had to be set manually rather than learned from data.

3. **medium Given a perceptron with weights $\omega_1=0.5$, $\omega_2=0.5$, threshold rule $f(\Sigma)=1$ if $\Sigma>0.6$ else 0, input $(x_1,x_2)=(1,1)$, target $y=1$, and learning rate $\alpha=0.2$, compute one weight update step.**

$\Sigma = 0.5 \cdot 1 + 0.5 \cdot 1 = 1.0 \rightarrow f(\Sigma)=1 = y$, so $\varepsilon = y - f(\Sigma) = 0 \rightarrow$ **no update** (weights stay 0.5, 0.5), since the sample is already correctly classified.

4. **medium Same setup as Q3 but input $(x_1,x_2)=(1,0)$, target $y=1$. Compute the updated weights.**

$\Sigma = 0.5 \cdot 1 + 0.5 \cdot 0 = 0.5 \rightarrow f(\Sigma)=0$ (since 0.5 is not > 0.6), $y=1 \rightarrow \varepsilon = 1 - 0 = 1$. Update: $\omega_1 \leftarrow 0.5 + 0.2 \cdot 1 \cdot 1 = 0.7$; $\omega_2 \leftarrow 0.5 + 0.2 \cdot 1 \cdot 0 = 0.5$ (unchanged, since $x_2=0$).

5. **easy State the perceptron convergence guarantee, and its precondition.**

If the training data is linearly separable, the perceptron learning rule is guaranteed to find a set of weights that correctly classifies all training samples in a finite number of updates. If the data is not linearly separable (e.g. XOR), the algorithm will never converge.

6. **easy Why can't a single-layer perceptron represent XOR?**

XOR's positive class occupies two diagonally opposite corners of the input square, which cannot be separated from the negative class by any single straight line (linear decision boundary) — the function is not linearly separable.

7. **medium Sketch (conceptually) how a 2-neuron hidden layer plus one output neuron can compute XOR from OR and NAND. What logical operation must the output neuron perform?**

Hidden neuron 1 computes $OR(x_1, x_2)$, hidden neuron 2 computes $NAND(x_1, x_2)$ (equivalently NOT-AND); the output neuron then computes the AND of these two hidden outputs: $AND(OR(x_1, x_2), NAND(x_1, x_2)) = XOR(x_1, x_2)$. This shows how composing two linear boundaries plus non-linear activations builds a non-linear function.

8. **easy What year and paper introduced backpropagation, and what mathematical rule does it apply efficiently?**

1986, Rumelhart, Hinton & Williams, "Learning representations by back-propagating errors" (Nature). It efficiently applies the (multivariate) Leibniz chain rule (1673) to compute gradients of the loss with respect to every weight in a feedforward network.

9. **hard For $L = 3y^2$, $y = x+2$, derive dL/dx using backpropagation-style forward/backward passes.**

Forward: $y = x+2 \rightarrow L = 3y^2$. Backward: $\partial L/\partial y = 6y = 6(x+2)$. $\partial y/\partial x = 1$. Chain rule: $\partial L/\partial x = \partial L/\partial y \cdot \partial y/\partial x = 6(x+2) \cdot 1 = 6x+12$.

10. **hard If a hidden unit y feeds into two downstream computations $u(y)$ and $v(y)$ which are combined into loss $L(u,v)$, why must you sum the two backpropagated paths when computing $\partial L/\partial y$, and what would go wrong if you didn't?**

This follows from the multivariate chain rule: $dL/dy = (\partial L/\partial u)(du/dy) + (\partial L/\partial v)(dv/dy)$ — every distinct forward path from y to L contributes independently to how a small change in y affects L , so the total sensitivity is the sum of the sensitivities along each path. Omitting one term would under-count y 's true influence on the loss, producing an incorrect (too small) gradient and hence incorrect weight updates.

11. **easy List four commonly used activation functions and one key property of each.**

Threshold (binary, non-differentiable); Sigmoid (smooth, range (0,1), vanishing gradients); Tanh (smooth, zero-centered, range $(-1,1)$); ReLU ($\max(0,x)$, cheap, mitigates vanishing gradients but can "die").

12. **medium Why is the sigmoid activation particularly prone to the "vanishing gradient" problem, and how does**

this affect training of deep networks?

Sigmoid saturates (flattens) for large positive or negative inputs, so its derivative approaches 0 there. During backpropagation, gradients are multiplied layer by layer (chain rule); if many layers have near-zero local derivatives, the product shrinks exponentially with depth, so gradients reaching early layers become vanishingly small — those layers effectively stop learning.

13. **medium** What is the "dying ReLU" problem?

If a ReLU unit's weighted input is consistently negative (e.g., due to a bad initialization or large negative update), its output and gradient are both 0 for all inputs, so it never activates or updates again — it "dies" and stops contributing to the network.

14. **easy** What does the loss function measure, and what is the goal of training with respect to it?

The loss function quantifies the discrepancy between the network's predicted output and the true target for given inputs. Training aims to find parameters w that minimize this loss (ideally over both training and — via generalization — unseen data).

15. **easy** Write the gradient descent update rule and explain each symbol.

$w(t+1) = w(t) - \alpha \nabla_w L(y, X, w)$. $w(t)$: current parameters; α : learning rate (step size); $\nabla_w L$: gradient of the loss with respect to w , pointing in the direction of steepest increase (hence the minus sign to descend).

16. **medium** Explain, using a diagram-style description, what happens with a learning rate that is (a) too low, (b) well-tuned, (c) too high.

(a) Too low: steps are tiny, so the loss decreases extremely slowly, wasting compute/time. (b) Well-tuned: steps move steadily and efficiently toward the minimum. (c) Too high: steps overshoot the minimum, potentially bouncing to a higher-loss point on the other side, causing oscillation or outright divergence (loss increases over iterations).

17. **easy** Distinguish underfitting from overfitting in terms of training vs. test error.

Underfitting: high error on both training and test data (model too simple / undertrained). Overfitting: low training error but high test error (model has memorized training-set-specific noise/patterns rather than generalizable structure).

18. **easy** Name four techniques to reduce overfitting and briefly explain one in more detail.

Training on more data, data augmentation, dropout, early stopping, regularization (L1/L2). Example detail — Dropout: during training, randomly "turns off" a fraction of neurons on each forward pass, forcing the network to not rely too heavily on any single neuron/co-adapted feature group, which improves robustness/generalization at test time (when all neurons are active, typically with rescaled outputs).

19. **medium** What is L2 regularization also known as, and what does it add to the loss function conceptually?

Also known as weight decay, ridge regression, or Tikhonov regularization. It adds a penalty term proportional to the sum of squared weights to the loss function, discouraging the model from assigning very large weights to any single feature/connection, which tends to produce smoother, less overfit models.

20. **hard** Formally, how does Goodfellow et al. (2016) define "regularization," and why is it notable that the definition targets generalization error specifically (not training error)?

Regularization is defined as any modification to a learning algorithm intended to reduce its generalization error but not (necessarily) its training error. This is notable because it explicitly acknowledges that regularization may make the model fit the training data slightly worse — the point is not to minimize training error at all costs, but to produce a model that performs well on new, unseen data, which is the actual objective of learning.

21. **easy** What is the practical difference between "training" and "inference" in terms of which passes (forward/backward) are executed?

Training executes both a forward pass (to compute the prediction and loss) and a backward pass (to compute gradients and update weights). Inference executes only the forward pass, since no learning/weight update is needed — just producing a prediction for new input.

22. **medium** Why are GPUs particularly well suited to deep learning workloads?

Deep neural network computation is dominated by large matrix/tensor multiplications (in both forward and backward passes); GPUs are architected for massive parallelism and are highly efficient at exactly these matrix operations, which allows training of large networks on large datasets within practically feasible time — a key enabling factor for the deep learning "renaissance."

23. **medium** Why might X_{batch} (a mini-batch) be strictly smaller than the full dataset X during training, beyond just memory constraints?

Besides GPU memory limits, using smaller batches introduces stochastic noise into the gradient estimate, which can help the

optimizer escape sharp local minima or saddle points and has been empirically observed to improve generalization compared to full-batch gradient descent; it also allows more frequent weight updates per epoch.

24. easy What historical event is often attributed to the publication of "Perceptrons" by Minsky and Papert (1969), and why?

It is often cited as a contributing cause of the first "AI winter" — a period of reduced funding and interest in neural network research — because the book rigorously proved the single-layer perceptron's inability to solve non-linearly-separable problems like XOR, and no practical algorithm for training multi-layer networks existed yet at the time.

25. medium In the neuron diagram $f(w_1x_1 + w_2x_2 + b)$, what is the specific role of the bias term b , distinct from the weights?

While weights scale the relative importance/contribution of each input, the bias b is an input-independent offset that shifts the activation function's effective threshold — allowing the neuron to output a non-zero value (or activate) even when all inputs are zero, and giving the decision boundary more flexibility (it need not pass through the origin).

26. hard Compare the perceptron learning rule (1958) with gradient descent + backpropagation (1986) as two distinct approaches to "learning weights." What key limitation of the earlier rule does the later approach overcome?

The perceptron rule updates weights directly based on the sign/magnitude of the classification error for a single-layer, threshold-activation model, and only works for linearly separable problems. Gradient descent with backpropagation generalizes this to arbitrarily deep, multi-layer networks with differentiable (non-threshold) activations, computing exact gradients of a general loss function with respect to every weight via the chain rule — overcoming both the single-layer restriction and the requirement of linear separability, at the cost of needing differentiable activation functions.

27. easy What does "feedforward" mean in the context of a Multi-Layer Perceptron?

Information flows in one direction only — from the input layer, through the hidden layers, to the output layer — with no cycles or feedback connections back to earlier layers.

28. medium Describe the supervised learning setup notation $X = \{(x_i, y_i)\}_{i=1}^N$ used in the lecture, and explain each component.

X denotes the full labeled training dataset, consisting of N (input, label) pairs. Each $x_i = (x_{i1}, \dots, x_{ip})$ is a p -dimensional feature vector for the i -th example, and y_i is its corresponding true label/target. The network is trained to learn a mapping from x_i to y_i across all N examples.

29. hard Explain why "choice of output layer, activation function, and loss function depends on the task," giving one example each for regression and multi-class classification.

Different tasks require outputs with different mathematical structure and correspondingly different loss functions to be meaningful. Regression: a single linear (identity) output unit (unbounded real-valued output) paired with a loss like mean squared error, since predictions and targets are continuous real numbers. Multi-class classification: a softmax output layer (producing a probability distribution over C classes) paired with a cross-entropy / negative-log-likelihood loss, since predictions must be interpretable as class probabilities summing to 1.

30. hard Synthesize: trace the full historical arc from 1943 to 1986 covered in this lecture, explaining what specific bottleneck each milestone resolved (or created).

1943 (McCulloch-Pitts): established a mathematical neuron model, but with fixed, hand-set weights — no learning. 1957/58 (Rosenblatt): introduced an algorithm to learn weights automatically from data via error-driven updates — but only for a single layer with linearly separable data. 1969 (Minsky-Papert): rigorously proved this single-layer limitation (XOR problem), stalling progress since no training method existed yet for multi-layer networks (an "AI winter" followed). 1986 (Rumelhart, Hinton, Williams): introduced backpropagation, an efficient application of the chain rule enabling gradient-based training of deep, multi-layer, differentiable-activation networks — finally overcoming the linear-separability bottleneck and enabling the modern deep learning era.

11. Formula & Definition Cheat Sheet

Concept	Formula / Definition
McCulloch–Pitts neuron	$y = 1$ if $\sum_i w_i x_i > \text{threshold}$, else 0 (weights fixed, not learned)
Perceptron learning rule	$w_i(n+1) \leftarrow w_i(n) + \alpha \cdot (y_i - \theta_i) \cdot x_i$
Neuron (MLP)	$y = f(\sum_i w_i x_i + b)$
Threshold activation	$f(x) = 1$ if $x > 0$, else 0
ReLU	$f(x) = \max(0, x)$
Sigmoid	$f(x) = 1/(1+e^{-\beta x})$
Tanh	$f(x) = (e^x - e^{-x})/(e^x + e^{-x})$
Gradient descent update	$w^{(t+1)} = w^{(t)} - \alpha \nabla_w L(y, X, w)$
Chain rule	$d/dx f(g(x)) = (df/dg)(dg/dx)$
Multivariate chain rule	$d/dx f(g_1, \dots, g_M) = \sum_i (\partial f / \partial g_i) (dg_i/dx)$ — sum over all paths!
Dataset gradient decomposition	$\nabla_w L(y, X, w) = \sum_i \nabla_w L(y_i, x_i, w)$
Regularization (Goodfellow)	Any modification to reduce generalization error, not (necessarily) training error

Key Names & Years

Year	Author(s)	Contribution
1673	Leibniz	Chain rule
1847	Cauchy	Gradient descent
1943	McCulloch & Pitts	First artificial neuron (no learning)
1957/58	Rosenblatt	Perceptron learning rule
1969	Minsky & Papert	Proved linear-separability limit (XOR)
1986	Rumelhart, Hinton, Williams	Backpropagation

Section-by-Section Quick Recap

- **AI $\supset$ ML $\supset$ DL:** DL = ML with deep neural nets, auto feature learning.
- **Biological neuron:** dendrites→inputs, axon→output, threshold→activation.
- **Single-layer perceptron:** linear boundary only; learns via error-driven updates.
- **XOR problem:** not linearly separable → needs hidden layer(s).
- **MLP:** feedforward, fully-connected, needs non-linear activations.
- **Activation functions:** threshold (non-diff.) → sigmoid/tanh (vanishing grad.) → ReLU (cheap, standard).
- **Loss + gradient descent:** minimize $L(y, X, w)$ by stepping opposite the gradient.
- **Backpropagation:** forward = values, backward = chain-rule gradients; sum at fan-out nodes.
- **Learning rate:** too low = slow; too high = diverges; tune carefully.
- **Overfitting:** low train / high test error → use more data, augmentation, dropout, early stopping, regularization.
- **Training vs inference:** training = forward+backward (expensive); inference = forward only (cheap).
- **GPUs:** enable large matrix ops at scale → made deep learning practical.

Final exam-prep reminder: Be ready to (1) hand-trace a perceptron weight update given numeric inputs, (2) draw/explain why XOR needs a hidden layer, (3) perform a small forward+backward pass computing gradients via the chain rule (watch for fan-out summing!), and (4) discuss the overfitting/regularization trade-off conceptually — these four skills cover the large majority of past-year exam question types for this lecture.